GRADUATION PROJECT DOCUMENTATION

CLARA

AI-Powered Accounting Office Automation System

Supervisor
Dr. Mohamed El Halaby

2026

Team members

Amr Yasser
Adham Ehab
Mohamed Anwar
Ahmed Ashraf Elkot
Sief Mohamed Saber
Ahmed Mohamed Yehia

Table of Contents

Table of Contents 1
Executive Summary 3
Solution Overview 3
Key Technical Contributions 3
Outcome 4
Chapter 1: Introduction 4
1.1 Project Background 5
1.2 Problem Statement 5
1.2.1 Task & File Management 5
1.2.2 Tracking Employee Work 5
1.2.3 Frequent Contract Updates 5
1.3 Project Objectives 6
1.4 Scope 6
1.5 Document Structure 6
Chapter 2: Problem Analysis 7
2.1 Problem 1 — Task Management 8
2.1.1 Bad Communication 8
2.1.2 No Dedicated Workspace 8
2.2 Problem 2 — Tracking Employees' Work 8
2.2.1 Hard to Track 8
2.2.2 Departments Multiply the Problem 8
2.2.3 Tracking Each User 8
2.3 Problem 3 — Documents & Contract Management 8
2.3.1 Archiving & Organization 8
2.3.2 Creating & Editing Contracts 9
2.3.3 The Contract Amendment Workflow in Detail 9
Chapter 3: Proposed Solution & System Modules 9
3.1 A Dedicated Task Management System 10
3.1.1 Employee View 10
3.1.2 Manager View 10
3.1.3 Notifications 10
3.2 An Interactive, Centralized Dashboard 10
3.2.1 Scale Metrics (Row A) 10

3.2.2 Activity & Engagement Metrics (Row B) 11
3.3 Employee Directory & HR Module 11
3.4 Documents & Contract Management Module 12
Chapter 4: System Architecture 12
4.1 High-Level Architecture 13
4.2 Multi-Tenancy & Organizational Model 13
4.3 Role-Based Access 13
4.4 Task Lifecycle 13
4.5 Notification Flow 14
4.6 Contract Processing Flow 14
4.7 Design Considerations 14
Chapter 5: AI Features 15
5.1 Motivation: The Contract Amendment Challenge 16
5.2 CORE: Clause-Level OCR & Retrieval 16
5.2.1 Dataset 16
5.2.2 Model 16
5.2.3 Training 17
5.2.4 Choose Model 17
5.2.4 Final Assembly 17
5.3 CAT: Contract Amendment Transformer 17
5.3.1 Dataset 17
5.3.2 Model 17
5.3.3 Training 18
5.5 Evaluation Methodology 18
5.6 Clause-Level RAG Retrieval 19
5.6.1 Why RAG Was Needed 19
5.6.2 Clause Labeling 19
5.6.3 Retrieval Pipeline 19
5.6.4 Retrieval Models 19
Chapter 6: Why CAT & CORE Instead of General AI? 20
6.1 Privacy & Security 21
6.2 Domain Knowledge 21
6.3 Document Understanding 21
6.4 Editing Precision 21
6.5 Operational Cost 22

6.6 Consistency 22
Chapter 7: Results, Limitations & Future Work 22
7.1 Current Status 23
7.2 Impact 23
7.3 Known Limitations 23
7.4 Future Work 24
7.5 Conclusion 24
Appendix A: Technical Appendix 24
A.1 Retrieval & Re-Ranking: Mathematical Foundation 25
A.2 Evaluation Metrics: Mathematical Foundation 25
A.2.1 ROUGE-L 25
A.2.2 Semantic Similarity 26
A.2.3 Edit Similarity 26
A.2.4 Change Accuracy 26
A.2.5 Final Weighted Score 26
A.3 CAT Dataset Structure 26
A.4 CAT Training Configuration Summary 27
A.5 CORE Pipeline Configuration Summary 28
A.6 Glossary 28

Executive Summary

CLARA is an AI-powered office automation system built to address the operational challenges that accounting and administrative offices face as they grow. The project was developed as a graduation project under the supervision of Dr. Mohamed El Halaby, and combines a full-stack web application with two purpose-built AI components for Arabic legal-contract processing.

Growing accounting offices typically rely on informal communication channels such as WhatsApp to assign and track work, have no centralized way to monitor employee activity across branches and departments, and spend hours manually editing legal contracts stored as paper or scanned documents. These three pain points — task management, employee tracking, and contract management — form the basis of CLARA's problem statement and directly shape its three core solution pillars.

Solution Overview

- A dedicated task management workspace that replaces ad-hoc chat-based coordination with structured tasks, comments, file attachments, approval workflows, and real-time notifications.
- An interactive, centralized dashboard giving managers company-wide visibility into active users, partner companies, recent tasks, and recent documents — broken down by branch and department.
- An AI-driven contract management pipeline that digitizes scanned Arabic legal contracts via OCR, structures their content, and uses a fine-tuned language model to perform targeted clause-level amendments in seconds instead of the 30–60 minutes typically required for manual editing.

Key Technical Contributions

- CORE (Clause-level OCR & Retrieval Engine): a three-phase pipeline that converts scanned Arabic contracts into structured, validated JSON using OCR, LLM-based structured extraction, and template-based correction.
- CAT (Contract Amendment Transformer): a LoRA fine-tuned Qwen3-4B model that takes a contract clause and a natural-language amendment request and returns a precise, structurally consistent updated clause.
- A clause-level Retrieval-Augmented Generation (RAG) layer using a multilingual dense retriever and a cross-encoder re-ranker, which lets the system operate on individual clauses rather than entire 10–20+ page contracts — reducing token cost and improving editing accuracy.

- A quantitative evaluation pipeline that blends four metrics — ROUGE-L, semantic similarity, edit similarity, and a numeric change-accuracy check — into a single weighted score used to benchmark candidate models objectively.

Outcome

By replacing a manual, error-prone, paper-driven workflow with a structured digital platform and a locally deployable, domain-specific AI pipeline, CLARA turns a process that previously supported roughly five contracts per day, at 30–60 minutes of manual editing each, into one capable of completing the equivalent amendment in seconds, while keeping all processing in-office for data privacy and avoiding recurring third-party AI subscription costs.

Chapter 1: Introduction

1.1 Project Background

Accounting and administrative offices are, by nature, document- and task-heavy organizations. As such offices grow — adding branches, departments, and clients — their internal coordination and document-handling processes tend to remain manual and informal, even as the volume of work scales. CLARA (AI-Powered Accounting Office Automation System) was conceived to digitize and streamline this operational layer, combining a conventional task- and document-management platform with AI features specifically engineered for Arabic-language legal contract processing.

1.2 Problem Statement

Accounting and administrative offices today struggle to keep operations organized as they grow. Three recurring problems were identified through analysis of how these offices currently operate:

1.2.1 Task & File Management

Work and files are scattered with no structured way to assign or track them. Employees and managers rely on WhatsApp chats, groups, or direct phone calls to communicate about tasks, which makes both communication and tracking significantly harder. Because there is no dedicated workspace, all task discussion spreads across chat history, making it difficult to see who is doing what and what has actually been completed. The outcome is that tasks fall through the cracks, deadlines slip, and managers cannot intervene in time.

1.2.2 Tracking Employee Work

Managers lack visibility into who is working on what, across branches and teams. As the number of branches grows, it becomes nearly impossible to track all ongoing work without a centralized dashboard, and each branch typically contains multiple departments, which multiplies the tracking challenge further. Managers need to know each user's current status — actively working, on leave, or no longer active — and what they are working on at any given moment.

1.2.3 Frequent Contract Updates

Legal contracts are the backbone of the business, yet offices hold large volumes of hard copies that are very difficult to manage and retrieve when needed. Updating contracts is slow and manual, often consuming 30–60 minutes of staff time per amendment once the relevant paper contract has been located, located clauses have been manually searched, and edits have been applied by hand — a process that is both time-consuming and error-prone, and which limits offices to processing roughly five contracts per day.

> *Result: poor communication, wasted time, and management overhead that slows the whole office down.*

1.3 Project Objectives

CLARA was designed with the following objectives:

1. Provide a dedicated task management workspace that structures communication between managers and employees and replaces informal chat-based coordination.
2. Provide managers with a centralized, real-time dashboard summarizing company-wide activity across users, companies, tasks, and documents.
3. Provide a directory and profile system for tracking each employee's role, department, status, and history of assigned and completed work.
4. Digitize legal contract archives by converting scanned Arabic documents into structured, machine-readable data using OCR and information extraction.
5. Automate contract amendment using a fine-tuned, domain-specific language model that edits only the relevant clause while preserving legal structure and consistency.
6. Keep all AI processing local to the office for data privacy and to avoid recurring third-party API costs, while maintaining accuracy comparable to general-purpose AI assistants.

1.4 Scope

The system scope covers two integrated layers: (1) a web-based office-management platform covering task management, employee/HR directories, company records, document storage, and a contracts module, and (2) an AI subsystem dedicated to contract digitization (CORE) and contract amendment (CAT), supported by a clause-level retrieval layer (RAG) and a quantitative evaluation pipeline used during model development. The AI subsystem specifically targets Arabic-language legal contracts, reflecting the target market of accounting and legal offices operating in Arabic-speaking jurisdictions.

1.5 Document Structure

This documentation is organized as follows:

- Chapter 2 — details the problems being solved, organized by problem area, with supporting context from the system's design phase.
- Chapter 3 — describes the proposed solution and the system's functional modules (task management, dashboard, HR/employee directory, and contract management).
- Chapter 4 — covers the system architecture and the underlying technology choices.

- Chapter 5 — presents the AI subsystem in depth: CORE, CAT, the RAG retrieval layer, model selection, and the evaluation methodology.
- Chapter 6 — discusses why a custom AI pipeline was chosen over general-purpose AI assistants.
- Chapter 7 — covers results, limitations, and future work.
- Appendix A — a technical appendix with implementation-level detail: mathematical formulations, model configurations, dataset structure, and pipeline parameters.

Chapter 2: Problem Analysis

This chapter expands on the three problem areas introduced in Chapter 1, examining each in terms of its root cause and operational impact, before Chapter 3 maps each problem to a corresponding solution module.

2.1 Problem 1 — Task Management

2.1.1 Bad Communication

Employees and managers rely on WhatsApp chats, groups, or direct calls to coordinate work. This makes communication and tracking significantly harder, since there is no single source of truth for what was asked, who it was assigned to, or what the current status is.

2.1.2 No Dedicated Workspace

All task discussion spreads across chat history. This makes it hard to see who is doing what and what is actually done, because progress updates, file attachments, and approvals are interleaved with unrelated conversation and are not tied to a discrete, trackable task record.

> *Outcome: tasks fall through the cracks, deadlines slip, and managers can't intervene in time.*

2.2 Problem 2 — Tracking Employees' Work

2.2.1 Hard to Track

Many branches make it nearly impossible to track all work without a centralized dashboard. As branch count grows, the manual effort required to stay informed about company-wide activity grows with it.

2.2.2 Departments Multiply the Problem

Each branch has multiple departments, which multiplies the tracking challenge further — a manager now has to reason about activity along two dimensions (branch and department) rather than one.

2.2.3 Tracking Each User

Managers need to know each user's status: working, on holiday, or no longer active — and what they are working on right now. Without structured employee records, this status information lives only in informal knowledge or is altogether unavailable.

2.3 Problem 3 — Documents & Contract Management

2.3.1 Archiving & Organization

Legal contracts are the backbone of the business, but offices hold large volumes of hard copies that are very difficult to manage and retrieve when needed. Locating a specific contract, or a specific clause within it, can itself consume significant staff time.

2.3.2 Creating & Editing Contracts

New contracts are created frequently and can take hours each, including time spent searching through past hard copies for reference language. Doing this by hand is a major time sink and is prone to human error — particularly when amendments must preserve specific numerical values (durations, capital amounts, addresses) embedded throughout the document.

2.3.3 The Contract Amendment Workflow in Detail

The end-to-end manual amendment workflow observed in practice follows a consistent, costly pattern:

7. A company submits a request to amend an existing contract.
8. Staff retrieve the relevant paper contract, often 20+ pages long.
9. Staff manually search the document for the clause(s) affected by the request.
10. Staff manually edit the clause(s), a process taking approximately 30–60 minutes per contract.
11. The edit carries an inherent risk of human error, particularly around numerical fields.

At this rate, an office can process approximately five contracts per day, which the project identifies as:

- Time Consuming — each amendment ties up staff time that could be spent on higher-value work.
- Error Prone — manual editing of legal text risks inconsistencies, especially with numbers and cross-referenced metadata.
- Limited in Productivity — throughput is capped by how fast a person can search and edit text by hand.

> *So why make humans do such repetitive, time-consuming work — in the age of AI?*

This observation motivated the design of CORE and CAT (covered in Chapter 5), which together replace the manual search-and-edit steps with an intelligent OCR and retrieval pipeline and a fine-tuned amendment model.

Chapter 3: Proposed Solution & System Modules

CLARA addresses the three problem areas above through three corresponding solution modules, plus a supporting employee/HR directory module that underpins both task management and dashboard reporting.

3.1 A Dedicated Task Management System

CLARA introduces a specialized task management workspace between managers and employees to track work clearly and consistently. The workspace is organized around three views:

3.1.1 Employee View

Employees can add comments and attach files to communicate progress directly on the task, rather than through an external chat application. Each task tracks a title, description, assignee, due date, and a threaded comment history, as observed in the implemented interface (task detail view, Arabic UI: العنوان / الوصف / الموظف / موعد التسليم / التعليقات).

3.1.2 Manager View

Managers see assigned tasks and their status, can expand a task to review comments and attached files, and can accept or reject completed work with feedback. Tasks are organized into status columns (observed states: Pending, In Progress, Completed) consistent with a Kanban-style board, with counts of overdue tasks, completed tasks, and total assigned tasks surfaced at the top of the view.

3.1.3 Notifications

Both sides are notified instantly when a task they are engaged in is updated — for example, an employee receives a notification immediately when assigned a new task, closing the communication gap that previously required a separate WhatsApp message.

These features directly address the task-tracking problem identified in Section 2.1.

3.2 An Interactive, Centralized Dashboard

CLARA gives managers one clean dashboard for the whole company — active users, partner companies, recent tasks, and recent documents at a glance. The dashboard is structured in two rows of summary metrics, reflecting both organizational scale and current activity/engagement levels.

3.2.1 Scale Metrics (Row A)

Metric	Description
Total Tasks	Count of all tasks recorded in the system.
Active Companies	Number of partner/client companies currently being served.
Departments	Number of departments configured across the organization.
Total Users	Total number of registered employees and managers.

3.2.2 Activity & Engagement Metrics (Row B)

Metric	Description
Open Tasks	Tasks not yet marked complete.
Contracts Awaiting Approval	Draft contracts pending sign-off.
Branches	Number of branch offices being tracked.
Active Users	Users who have shown recent activity in the system.

Beneath the summary metrics, the dashboard surfaces three further panels: recent contracts (type, associated company, status, and creation date), recent tasks (title, status, assignee, and due date), and — in an illustrative scaled view used for presentation purposes — company-wide totals such as 248 active users, 32 active companies, 164 tasks processed in a week, and 57 documents created, demonstrating the dashboard's intended use at production scale.

Note: figures such as 248 / 32 / 164 / 57 are illustrative scale figures used in the project presentation; the live system displays real-time counts drawn from the office's own data.

3.3 Employee Directory & HR Module

Supporting both the task and dashboard modules, CLARA includes an employee directory (دليل الموظفين) that tracks, per employee:

- Job title and department/branch assignment.
- Employment status — Active, On Leave, or Inactive.
- Hire date, contact information (email, phone), and national ID.

- Tasks assigned by the employee, tasks assigned to the employee, and contracts the employee has authored.
- Account verification status (e.g., Pending Verification) for newly added staff.

Managers can search and filter the directory by name/email, department, and status, and can add new employees directly from the dashboard. Role-based access is reflected in the system through distinct role badges observed in the interface, including Owner (OW), Head of Branch (HE), Employee (EM), and Admin (AD).

3.4 Documents & Contract Management Module

The contracts module stores and tracks legal documents per partner company, including contract type (e.g., founding/establishment contracts), status (e.g., Draft), associated company, and creation timestamp. This module is the entry point for the AI-powered contract digitization and amendment pipeline detailed in Chapter 5.

Chapter 4: System Architecture

CLARA is composed of two cooperating layers: a web-based office-management application (the system of record for tasks, employees, companies, and documents) and an AI subsystem dedicated to contract digitization and amendment. This chapter describes the architecture at the level observable from the system's design and interface; implementation-level technical detail for the AI subsystem is provided in Chapter 5 and Appendix A.

4.1 High-Level Architecture

At a high level, the platform is organized into the following logical layers:

12. Presentation Layer — a role-aware web interface (observed in both Arabic and English) providing dashboard, task, HR/employee, and contract views distinct for employees, managers, and owners.
13. Application / Business Logic Layer — manages task lifecycle (Pending → In Progress → Completed), notifications, role-based permissions, and aggregation logic for dashboard metrics.
14. Data Layer — persists organizational data: users, branches, departments, companies, tasks, comments, attachments, and contracts.
15. AI Subsystem — a separate processing pipeline (CORE + CAT + RAG, detailed in Chapter 5) invoked when a contract is digitized or amended, decoupled from the core application so it can be deployed and scaled independently.

4.2 Multi-Tenancy & Organizational Model

The system is structured around a multi-branch, multi-department organizational model. Each company (tenant) using CLARA — referred to in the interface as المحاسبة / "Accounting Office" — can configure multiple branches (الفروع), each of which contains multiple departments (الأقسام). Users are assigned to a department and branch, and dashboard aggregations roll activity up across this hierarchy. This directly addresses the "departments multiply the tracking challenge" issue raised in Section 2.2.

4.3 Role-Based Access

The interface exposes distinct role badges, observed as: Owner (OW), Head of Branch (HE), Employee (EM), and Admin (AD). Permissions differ by role — for example, employees can comment on and attach files to their own tasks, while managers/owners can additionally assign tasks, accept or reject submitted work, and access company-wide dashboard views.

4.4 Task Lifecycle

Tasks observed in the system move through the following states:

State	Meaning
PENDING (قيد الإنتظار)	Task created and assigned, not yet started.
IN_PROGRESS (قيد التنفيذ)	The employee has begun work on the task.
DONE / COMPLETED (مكتمل)	Task marked complete, pending or after manager review.
ACCEPTED (تم القبول)	The manager accepted the work done by the employee

Each task carries a due date and a remaining-time indicator (e.g., "7d left"), and the dashboard surfaces an overdue-task count separately from in-progress and completed counts, giving managers an at-a-glance signal of risk.

4.5 Notification Flow

When a task is created, assigned, commented on, or has its status changed, the system generates a notification for the relevant counterpart (employee or manager). Notifications are surfaced via a bell icon with an unread-count badge, and include a short message (e.g., "You have been assigned a new task: test 5") consistent with an event-driven notification design.

4.6 Contract Processing Flow

The contract module integrates with the AI subsystem as follows: a scanned contract is ingested and OCR'd into structured JSON by CORE; the resulting structured contract, along with its clause breakdown, is stored and made available in the contracts module; when an amendment request is submitted, the RAG retrieval layer identifies the relevant clause, CAT generates the amended clause, and the result is merged back into the contract record. This flow is described in full in Chapter 5.

4.7 Design Considerations

- Separation of concerns — the AI pipeline operates independently of the core CRUD application, allowing the two to be scaled, deployed, and iterated on separately.
- Bilingual interface — the production interface is Arabic-first (right-to-left), matching the target user base of Arabic-speaking accounting and legal offices, while documentation and model development artifacts are maintained in English.

- Local deployment of AI components — the CAT/CORE pipeline is designed to run on office-local hardware rather than a third-party cloud API, addressing data-privacy concerns inherent to handling client legal contracts (expanded on in Chapter 6).

Chapter 5: AI Features

This chapter presents the AI subsystem that turns hours of manual contract work into seconds. It covers the motivation for an intelligent pipeline, the two core models built for the project (CORE and CAT), the retrieval layer that connects them, the process used to select the underlying language model, and the evaluation methodology used to benchmark results.

5.1 Motivation: The Contract Amendment Challenge

As detailed in Section 2.3.3, the manual contract amendment workflow follows a fixed, costly sequence: a company submits a request, staff retrieve a paper contract (often 20+ pages), staff manually search for the relevant clause, and staff manually edit it — a process taking 30–60 minutes and carrying a real risk of human error. At this rate, an office can process roughly five contracts per day. This workflow is time-consuming, error-prone, and limits productivity, motivating the need for intelligent OCR and automated contract amendment.

5.2 CORE: Clause-Level OCR & Retrieval

CORE's goal is to turn scanned Arabic contracts into structured JSON (fields and articles) despite OCR noise and long, repetitive boilerplate text typical of legal documents. CORE is implemented as a three-phase pipeline.

5.2.1 Dataset

- Pre-processing
  - 32 pdf file (30 train, 2 test)
  - Each file is converted to jpg photos and groping them in chunks each chuck of size 2 photos with sliding window technique (chunk1: [jpg1, jpg2], chunk2: [jpg2, jpg3], …)
- Total Number of sample points (chunks): 440
- Total Number of sample points after Augmentation: 1640
- Point structure: {

  chunk_i: [jpg_i, jpg_j]

  metadata: // details in Appendix A

  Articles: [ ] // // details in Appendix A

  }

5.2.2 Model

- Qwen-3VL-4B

5.2.3 Training

- Framework: Unsloth.
- Technique: LoRA (Low-Rank Adaptation) fine-tuning.
- Chosen for faster training, lower VRAM usage, and overall cost-efficiency relative to full fine-tuning.

5.2.4 Choose Model

- Human Judge: observe the results of the models after training process and choose the best one

5.2.4 Final Assembly

The final assembly step merges the overlapping chunks produced in Phase 1, drops partial duplicate segments created by the chunk overlap, and keeps complete segments to reconstruct a single coherent structured document. As of the latest project status, all three phases have been built and the 110-template library has been approved.

5.3 CAT: Contract Amendment Transformer

CAT is the model responsible for actually performing an amendment: given a contract clause and a natural-language amendment request, it returns an action title and the updated clause, while preserving legal structure and internal consistency.

5.3.1 Dataset

- 420 training samples and 45 test samples (see Appendix A for the dataset's full split and field structure, including a later 465-sample (375-train, 45-val, 45-test) configuration used during refinement).
- Each sample pairs a prompt (clause + amendment request) with the corresponding before/after clause.
- Point structure : {

  action_title : “string”,

  before: [“list of strings”],

  after: [“list of strings”],

  prompt : “string”

  }

5.3.2 Model

- Base model: Qwen3-4B, 4-bit quantized.
- Selected for its strong Arabic-language understanding.
- Enables efficient fine-tuning with comparatively low hardware requirements.

5.3.3 Training

- Framework: Unsloth.
- Technique: LoRA (Low-Rank Adaptation) fine-tuning.
- Chosen for faster training, lower VRAM usage, and overall cost-efficiency relative to full fine-tuning.

The CAT pipeline can be summarized as a simple transformation:

Contract Clause → CAT → Amended Clause

5.5 Evaluation Methodology

To compare candidate models objectively, every fine-tuned model's prediction is compared against a gold reference clause using four metrics, computed in parallel and then blended into a single 0–100 final score.

Metric	What It Measures	Method
ROUGE-L	Text similarity	rouge-score (Google) — LCS-based overlap.
Semantic Similarity	Meaning match	Qwen3-embeddings-0.6B, cosine similarity.
Edit Similarity	Character-level match	difflib SequenceMatcher (Ratcliff/Obershelp).
Change Accuracy	Number match	Regex + digit normalizer checking that key numbers were reproduced correctly.

These four metrics are blended with the following weights (illustrated visually in the original presentation as a weighted pie: Change Accuracy 45%, Semantic Similarity 30%, ROUGE-L 15%, Edit Similarity 10%), with the precise formula depending on whether the gold reference clause contains numerical values:

- Final = 0.45·Change Accuracy + 0.30·Semantic Similarity + 0.15·ROUGE-L + 0.10·Edit Similarity (when the gold clause contains numbers)
- Final = 0.45·Semantic Similarity + 0.35·ROUGE-L + 0.20·Edit Similarity (when the gold clause contains no numbers)

This dual formulation ensures that for clauses where getting a number right (e.g., a contract duration or capital amount) is critical, numerical accuracy dominates the score; for purely descriptive clauses, semantic and textual similarity take precedence. The full mathematical definitions of each metric are given in Appendix A.2.

5.6 Clause-Level RAG Retrieval

5.6.1 Why RAG Was Needed

Two problems arise from feeding an entire contract to a language model at amendment time:

- Challenge: contracts may run 10–20+ pages, and sending the full contract raises token cost while a large context window can hurt editing accuracy.
- Solution: split the contract into individual clauses, retrieve only the clause tied to the amendment request, send just that target clause to CAT, and merge the result back into the full contract.

5.6.2 Clause Labeling

Each clause is linked to contract metadata — company name, duration, capital, address, and business activities — so that CLARA can detect when an amendment affects this metadata and keep the structured JSON representation in sync with the amended clause text.

5.6.3 Retrieval Pipeline

User Request → RAG → Target Clause → Amendment Model (CAT) → Updated Contract

5.6.4 Retrieval Models

Component	Model	Role
Dense Retriever	Qwen3-embedding-0.6B	Converts Arabic clauses into dense semantic vectors; stored and searched with FAISS.
Re-Ranker	Qwen3-reranker-0.6B	Cross-encoder that re-ranks top retrieved clauses by evaluating the query and clause jointly, boosting precision beyond embedding search alone.

The mathematical foundation of the retrieval and re-ranking process — embedding, normalization, cosine similarity, top-K retrieval, and re-ranking — is presented in full in Appendix A.1.

Chapter 6: Why CAT & CORE Instead of General AI?

A natural question is why the project built dedicated models (CAT and CORE) rather than simply calling a general-purpose AI assistant such as ChatGPT or Gemini through an API. The comparison below summarizes the reasoning behind that decision.

Dimension	General AI (ChatGPT / Gemini)	CAT & CORE (CLARA's System)
Privacy & Security	External cloud processing	Local office deployment
Domain Knowledge	General purpose only	Accounting & contract expertise
Document Understanding	Requires entire contract context	Clause-level retrieval (CORE)
Editing Precision	May rewrite unrelated content	Targeted, precise amendments
Operational Cost	Pay per request / subscription	One-time deployment cost
Consistency	Output format may vary	Standardized amendment process

6.1 Privacy & Security

Legal contracts contain sensitive client information — company ownership structures, capital amounts, addresses, and business terms. Sending this data to an external cloud API introduces a privacy exposure that a locally deployed model avoids entirely, since processing never leaves the office's own infrastructure.

6.2 Domain Knowledge

A general-purpose model has broad but shallow knowledge. CAT is fine-tuned specifically on contract clause amendments, and CORE's template-correction phase is built directly from 32 ground-truth contracts — giving the system accounting- and contract-specific expertise that a general assistant does not have out of the box.

6.3 Document Understanding

Asking a general model to amend a clause inside a 20-page contract typically requires sending the entire document as context, which is both expensive in tokens and prone to degraded accuracy as context length grows. CORE's clause-level retrieval means only the relevant clause, plus its linked metadata, needs to be sent to the amendment model.

6.4 Editing Precision

General-purpose assistants, when asked to “edit a contract,” may inadvertently rewrite unrelated content, alter phrasing style, or introduce subtle wording drift elsewhere in the document. CAT is trained specifically to apply targeted, precise amendments to only the clause in question, preserving everything else verbatim.

6.5 Operational Cost

General AI APIs are billed per request or via subscription, meaning costs scale directly with usage volume — a significant factor for an office processing many contracts. CAT and CORE, once deployed, carry a one-time deployment cost rather than a recurring per-request fee.

6.6 Consistency

General-purpose model output format and style can vary between requests. CAT and CORE are designed around a standardized amendment process with a fixed output structure (action title + updated clause), making downstream integration with the contracts module predictable and reliable.

Chapter 7: Results, Limitations & Future Work

7.1 Current Status

As of the latest project milestone:

- The task management module, employee directory, and centralized dashboard are implemented and demonstrated, with live screens for employee task boards, task detail/comment views, manager task histories, notifications, the company-wide dashboard, employee directory, and individual employee profiles.
- CORE's three OCR/extraction/correction phases are all built, with a 110-template correction library approved against 32 ground-truth contracts.
- CAT has been fine-tuned on a 420-sample (refined to 465-sample) Arabic clause-amendment dataset using LoRA via Unsloth, on a Qwen3-4B (4-bit) base model selected through systematic benchmarking against Llama, Gemma, and multiple Qwen2.5/Qwen3 variants.
- A clause-level RAG layer (multilingual-e5-base retriever + bge-reranker-v2-m3 re-ranker) connects CORE's structured output to CAT's amendment capability.
- A four-metric evaluation pipeline (ROUGE-L, semantic similarity, edit similarity, change accuracy) is in place to benchmark model candidates on a held-out test set.

7.2 Impact

The combined system targets a substantial reduction in the time required to amend a contract clause — from the 30–60 minutes typical of manual editing (Section 2.3.3) down to a matter of seconds for the model-assisted amendment step, while keeping processing local to the office for data privacy and avoiding recurring third-party AI subscription costs (Chapter 6). On the operational side, the task management and dashboard modules replace informal, chat-based coordination with a structured, auditable workflow, directly addressing the visibility and communication gaps identified in Chapter 2.

7.3 Known Limitations

- GPU VRAM constrained the scale of model experiments; larger variants such as Qwen3-8B were identified as promising but not fully explored due to hardware limits.
- CORE's Phase 2 (structured extraction) still inherits OCR typos from Phase 1 in cases the Phase 3 template-correction step does not catch, particularly for contracts that deviate substantially from the 110 approved templates.
- The CAT training dataset, while purpose-built, is modest in size (several hundred samples); broader coverage of clause types and amendment phrasings would likely improve generalization.

- The evaluation pipeline benchmarks clause-level amendment quality but does not yet capture end-to-end user-facing metrics such as time saved per real-world amendment request or user satisfaction.

7.4 Future Work

16. Explore larger model variants (e.g., Qwen3-8B) as additional GPU VRAM becomes available, to evaluate whether amendment accuracy improves meaningfully.
17. Expand the CAT training dataset with a wider variety of clause types, amendment phrasings, and edge cases (e.g., multi-clause amendments in a single request).
18. Expand the CORE template library beyond 110 templates to cover a broader range of contract formats and legal document types beyond founding/establishment contracts.
19. Introduce end-to-end production metrics (e.g., real measured time-per-amendment, user acceptance/rejection rate of AI-suggested edits) to validate the system's impact under live office usage.
20. Extend the RAG and amendment pipeline to other document types relevant to accounting offices beyond legal contracts, such as financial statements or compliance filings.

7.5 Conclusion

CLARA demonstrates that a domain-specific, locally deployable AI pipeline can meaningfully outperform a general-purpose AI assistant on a narrow, high-value task — Arabic legal contract amendment — while simultaneously solving the everyday operational problems (task tracking, employee visibility, and document management) that motivated the project in the first place. By combining a conventional, well-structured office-management platform with a carefully engineered, evaluated, and benchmarked AI subsystem, the project shows a practical path for accounting and legal offices to modernize their workflows without sacrificing data privacy or incurring open-ended AI subscription costs.

Appendix A: Technical Appendix

This appendix collects the implementation-level detail referenced from Chapter 5: the mathematical formulation of the retrieval pipeline and evaluation metrics, dataset structures, and model configuration parameters.

A.1 Retrieval & Re-Ranking: Mathematical Foundation

Given a user's amendment request as a query q and a contract's clauses $d_1, d_2, \ldots, d_n$, the RAG pipeline (Section 5.5) identifies the target clause d* through the following steps:

Step	Formula	Description
1. Embed clauses	$v_i = f(d_i)$	Each clause d_i is mapped to a dense vector v_i using the multilingual-e5-base encoder f.
2. Normalize	$\hat{v}_i = v_i / \|v_i\|, \quad \hat{q} = q / \|q\|$	Both clause and query vectors are L2-normalized.
3. Similarity	$S(q, d_i) = \hat{q}^T \cdot \hat{v}_i$	Cosine similarity between the normalized query and each clause vector.
4. Top-K Retrieval	$R = \text{TopK}(S(q, d_i))$	The K highest-scoring clauses are shortlisted as candidates.
5. Re-Ranking	$r_i = g(q, d_i)$	Each candidate is re-scored by the cross-encoder re-ranker g (bge-reranker-v2-m3), which evaluates the query and clause jointly rather than via independent embeddings.
6. Best Clause	$d^* = \text{argmax}(r_i)$	The candidate with the highest re-ranked score is selected as the target clause for amendment.

This two-stage retrieve-then-re-rank design is a standard pattern for improving retrieval precision: the dense retriever (Step 3–4) is fast and scales to many clauses, while the cross-encoder re-ranker (Step 5–6) is more computationally expensive but more accurate, and is only applied to the small shortlist of top-K candidates rather than the full clause set.

A.2 Evaluation Metrics: Mathematical Foundation

A.2.1 ROUGE-L

ROUGE-L is an F-score based on the Longest Common Subsequence (LCS) between the predicted and gold text:

$$F = 2PR / (P + R)$$

- P = LCS(pred, gold) / |pred| — precision: fraction of the prediction that overlaps with the gold text.
- R = LCS(pred, gold) / |gold| — recall: fraction of the gold text recovered by the prediction.

Implemented using the rouge-score library (Google).

A.2.2 Semantic Similarity

Measures meaning match using cosine similarity between sentence embeddings:

$$sim = (e_pred \cdot e_gold) / (\| e_pred \| \times \| e_gold \|)$$

Embeddings e_pred and e_gold are produced using multilingual-e5-base; the resulting cosine similarity is scaled by 100 to produce a 0–100 score.

A.2.3 Edit Similarity

Measures character-level match using Python's difflib SequenceMatcher (Ratcliff/Obershelp algorithm):

$$ratio = 2M / T$$

- M = number of matching characters between prediction and gold.
- T = total number of characters across both strings.

A.2.4 Change Accuracy

Checks specifically whether key numbers (e.g., contract duration, capital amount) were correctly reproduced or updated:

$$Recall = |Nums_pred \cap Nums_gold| / |Nums_gold| \times 100$$

Numbers are extracted from both prediction and gold text via regular expressions, with a digit-normalizer step to account for formatting differences (e.g., Arabic-Indic vs. Western digits, thousands separators).

A.2.5 Final Weighted Score

The four component metrics are combined into a single 0–100 final score, with the formula depending on whether the gold clause contains numerical values:

$$Final = 0.45 \cdot CHA + 0.30 \cdot SS + 0.15 \cdot ROUGE_L + 0.10 \cdot ES \quad (gold\ has\ numbers)$$

$$Final = 0.45 \cdot SS + 0.35 \cdot ROUGE_L + 0.20 \cdot ES \quad (no\ numbers)$$

Where CHA = Change Accuracy, SS = Semantic Similarity, ROUGE_L = ROUGE-L F-score, and ES = Edit Similarity.

A.3 CAT Dataset Structure

Two dataset configurations were referenced across the project's development: an initial 420-train / 45-test split, and a later refined configuration of 465 total samples.

Configuration	Total	Train	Validation	Test
Initial	465	420	—	45
Refined	465	375	45	45

Each dataset sample follows a consistent structure:

Field	Description
Action / Prompt	The natural-language amendment request describing the requested change.
Before	The original contract clause text prior to amendment.
After	The gold-standard amended clause text.
P (metadata)	Supplementary metadata linking the clause to contract-level fields (e.g., company name, duration, capital, address, business activities) used for consistency checks.

This structure was found to be the most effective and easiest to process during dataset design.

A.4 CAT Training Configuration Summary

Parameter	Value
Base model	Qwen3-4B
Quantization	4-bit
Fine-tuning technique	LoRA
Training framework	Unsloth
Training samples	420 (initial) / 375 (refined)
Validation samples	45 (refined configuration)
Test samples	45
Sampling temperature (benchmarking)	0.1

A.5 CORE Pipeline Configuration Summary

Phase	Model / Method	Key Parameters
1 — OCR	Chandra-OCR-2	Overlapping 2-page chunking; deskew/contrast/blank-page preprocessing; repetition guard.
2 — Structured Extraction	Qwen (4-bit)	"Exact copy" constraint to suppress hallucination.
3 — Template Correction	110 templates from 32 ground-truth contracts	Content-based article matching (numbering-independent); smart merge (corrects small diffs, protects large diffs).

A.6 Glossary

Term	Definition
CORE	Clause-level OCR & Retrieval Engine — the three-phase pipeline converting scanned Arabic contracts into structured JSON.
CAT	Contract Amendment Transformer — the fine-tuned model that performs clause-level amendments.
RAG	Retrieval-Augmented Generation — the technique of retrieving relevant context (here, a clause) before generation rather than processing an entire document.
LoRA	Low-Rank Adaptation — a parameter-efficient fine-tuning technique that updates a small number of additional parameters rather than the full model.
FAISS	Facebook AI Similarity Search — a library for efficient nearest-neighbor search over dense vectors.
Cross-encoder	A model architecture that jointly encodes a query and candidate document together (as opposed to encoding them separately), generally yielding higher precision at higher computational cost.
ROUGE-L	A text-similarity metric based on the longest common subsequence between two texts.
Quantization (4-bit)	A technique for reducing model memory/compute requirements by representing weights with lower numerical precision.